Organizacja zajęć warsztatowych (2026/27)

Sztuczna inteligencja w grach i rozszerzonej rzeczywistości

Zajęcia warsztatowe dla studentów IS na WFiIS AGH

Piotr A. Kowalski

Zajęcia warsztatowe służą wspólnemu rozwiązywaniu niewielkich problemów związanych z zastosowaniem metod inteligencji obliczeniowej w grach. Podczas każdego spotkania należy uruchomić rozwiązanie, przeprowadzić eksperyment i opisać wnioski. Poniższa instrukcja określa organizację pracy, zasady oddawania rezultatów i zakres pięciu proponowanych tematów.

Zasady pracy

1. Stałe zespoły. Zajęcia odbywają się w zespołach cztero- lub pięcioosobowych. Zespół wspólnie realizuje zadania i odpowiada za oddawane materiały. Każdy uczestnik powinien rozumieć zastosowaną metodę oraz potrafić wyjaśnić otrzymane wyniki.

2. Jeden temat na spotkanie. Wszystkie zespoły rozwiązują to samo zadanie, lecz każdy przygotowuje własną implementację lub własne uzupełnienia kodu, eksperymenty i wnioski. Porównywanie wyników oraz dyskusja między zespołami są częścią warsztatu.

3. Zespół prowadzący. Na każde spotkanie wyznaczany jest jeden zespół, nazywany dalej zespołem liderskim. Przygotowuje on problem, konspekt, materiały i działający punkt wyjścia do ćwiczenia. Przydział tematów ustalany jest wcześniej (na zajęciach I).

4. Przewodnik przy każdym zespole. Podczas realizacji zadania członkowie zespołu liderskiego rozdzielają się pomiędzy pozostałe zespoły. Każdy zespół pracuje więc we własnym składzie, powiększonym o jedną osobę pełniącą rolę przewodnika. Przed spotkaniem należy dopasować podział i obsadę tak, aby zapewnić po jednym przewodniku każdemu zespołowi.

5. Wspieranie samodzielnej pracy. Przewodnik wyjaśnia problem, pomaga rozpoznawać błędy, zadaje pytania i wspiera interpretację wyników. Nie wyręcza zespołu w wykonaniu zadania. Zespół liderski w czasie tego spotkania pełni rolę prowadzącą, zamiast pracować jako osobny zespół wykonawczy.

6. Aktywny udział. W obrębie zespołu należy podzielić zadania związane z implementacją, testowaniem, analizą i bieżącą dokumentacją. Role można zmieniać w trakcie spotkania. Prowadzenie notatek nie zwalnia z udziału w rozwiązywaniu problemu.

Czas pracy i prezentacja wyników

Około 10 minut — prezentacja zespołu liderskiego. Na początku spotkania proszę przedstawić cel, zadanie do wykonania, narzędzia, materiały startowe i oczekiwane wyniki oraz wyjaśnić sposób ich sprawdzenia.

90 minut — praca zespołów. Czas obejmuje eksperymenty, bieżące pisanie sprawozdania i oddanie materiałów. Zadania zakładają gotowe środowisko i szkielet programu; nie wymagają budowy pełnej gry ani treningu dużych modeli. Silnik gry jest fakultatywny.

5–6 minut na każdy zespół — demo na forum grupy. Proszę pokazać działające rozwiązanie, najważniejsze wyniki i krótki wniosek. Osobne slajdy nie są wymagane. Prezentacja liderska i dema odbywają się poza 90 minutami pracy; dla czterech zespołów całe spotkanie zajmie około 120–124 minut.

Oddawanie materiałów i ocenianie

Co oddaje zespół wykonujący zadanie

Z każdych zajęć zespół przesyła przez **MS Teams, w zadaniu przypisanym do danego spotkania**, to, co udało mu się wykonać: kod źródłowy lub notatnik, niezbędne małe pliki wynikowe oraz krótkie sprawozdanie. Materiały należy oddać do końca 90-minutowej części roboczej, przed rozpoczęciem pokazów. Należy oddać również rozwiązanie częściowe, wskazując jego aktualny stan i napotkane trudności.

Sprawozdanie powstaje w trakcie zajęć. Jego treści nie należy przygotowywać przed spotkaniem ani uzupełniać po zajęciach. Można wcześniej pobrać pusty szablon. Wyniki, obserwacje i wnioski należy wpisywać na bieżąco podczas eksperymentów; ostatnie minuty części roboczej należy przeznaczyć na uporządkowanie i przesłanie plików.

Proponowana objętość to **około 3 stron**, ewentualnie odpowiadająca jej część opisowa notatnika. Do sprawozdania nie należy przepisywać całego kodu. Wystarczą opis zmiany, tabela lub wykres wyników oraz uzasadnione wnioski.

Stały układ krótkiego sprawozdania

1. **Temat i autorzy:** numer zespołu, imiona i nazwiska uczestników, data oraz nazwisko przewodnika z zespołu liderskiego.

2. **Cel i wykonane zmiany:** opis badanego zagadnienia oraz uzupełnionych elementów kodu lub konfiguracji.

3. **Eksperyment:** parametry, scenariusze testowe, wersja modelu, jeżeli występuje, oraz ziarna losowości, jeśli mają znaczenie.

4. **Wyniki:** niewielka tabela, wykres lub przykłady działania, obejmujące również nieudane próby.

5. **Wnioski i ograniczenia:** co zadziałało, co nie zadziałało i jakie obserwacje to potwierdzają. Należy krótko wskazać wkład poszczególnych osób.

Przy odnośniku do repozytorium należy wskazać wersję kodu oddawaną na zajęciach, np. identyfikator commita. Pliki powinny zawierać krótką instrukcję uruchomienia; dużych wag modeli nie trzeba ponownie przesyłać, jeśli podano jednoznaczną nazwę i sposób ich uzyskania.

Co podlega ocenie

Zespoły wykonawcze: poprawność i zakres wykonanego rozwiązania, jakość eksperymentu, interpretacja wyników, czytelność bieżącego sprawozdania oraz rzeczywisty udział członków zespołu. Liczy się udokumentowana praca i rozumienie metody. Dodatkowe rozszerzenia są fakultatywne; pierwszeństwo ma rzetelne wykonanie zakresu podstawowego.

Zespół liderski: jakość przygotowanych materiałów, jasność prezentacji wprowadzającej i opisu problemu, dopasowanie zadania do czasu, poprawność przygotowanego rozwiązania oraz to, jak zadanie udało się przeprowadzić podczas warsztatu. Oceniana jest także pomoc udzielana zespołom i reagowanie na trudności. Wyniki uczestników stanowią informację o jakości przygotowania, z uwzględnieniem przyczyn napotkanych problemów.

Materiały przygotowawcze I prezentacja zespołu liderskiego trafiają na Teams przed spotkaniem. Pod jego koniec zespół dopisuje krótkie podsumowanie przebiegu: zrealizowany zakres, typowe trudności i potrzebne poprawki. Wskazuje także wkład swoich członków.

Przygotowanie spotkań i plan tematów

Zadania zespołu liderskiego przed spotkaniem

Proszę przygotować około 10-minutową prezentację wprowadzającą, opis zadania oraz konspekt obejmujący następnie 90 minut pracy i dema po 5–6 minut na zespół. W opisie proszę wskazać cel, dane wejściowe, oczekiwany rezultat i sposób sprawdzenia działania. Proszę oddzielić zakres podstawowy od rozszerzeń dla zespołów, które skończą wcześniej.

Proszę udostępnić **pakiet startowy**: kod z wyraźnie oznaczonymi miejscami do uzupełnienia, mały zestaw danych, instrukcję uruchomienia, ustalone wersje bibliotek oraz pusty szablon sprawozdania. Własne rozwiązanie referencyjne i wyniki testów proszę przygotować do weryfikacji przez prowadzącego przed zajęciami.

Proszę przeprowadzić próbę całego ćwiczenia z pomiarem czasu. Proszę sprawdzić instalację na typowym laptopie i działanie na CPU. Przed warsztatem proszę pobrać potrzebne modele. Dla tematów LLM proszę zapewnić dostęp do działającego modelu lokalnego lub wcześniej uzgodnionej usługi; uczestnicy nie powinni dopiero na zajęciach zakładać płatnych kont.

Pięć tematów warsztatowych

1. **Metaheurystyki:** dobór ekwipunku bohatera przy ograniczonej pojemności plecaka za pomocą symulowanego wyżarzania.

2. **Uczenie ze wzmocnieniem:** agent Q-learning znajdujący drogę do wyjścia z niewielkiego labiryntu.

3. **Sieci neuronowe i YOLO:** wykrywanie przedmiotu jako warunek zdobycia punktu w minigrze wykorzystującej obraz z kamery.

4. **Duże modele językowe:** rozmowa z kupcem w grze oraz zamiana wypowiedzi gracza na kontrolowaną propozycję działania.

5. **Bezpieczeństwo systemów gier:** próby manipulacji postacią LLM i ochrona stanu gry przed nieuprawnioną zmianą.

Opisane dalej narzędzia i parametry stanowią spójną propozycję realizacji. Zespół liderski może uzgodnić inne rozwiązanie z prowadzącym, jeżeli zachowa cel, porównywalny zakres i możliwość ukończenia zadania w ciągu 90 minut oraz zademonstrowania wyniku w 5–6 minut.

Dokumentacja pomocnicza

Materiały techniczne do przygotowania pakietów startowych:

- NumPy i obliczenia numeryczne: https://numpy.org/doc/
- Matplotlib i prezentacja wyników: https://matplotlib.org/stable/users/index.html
- Ultralytics YOLO i detekcja: https://docs.ultralytics.com/modes/predict/
- Ollama i odpowiedzi zgodne ze schematem: https://docs.ollama.com/capabilities/structured-outputs
- OWASP i prompt injection: https://genai.owasp.org/llmrisk/llm01-prompt-injection/

Warsztat 1 Dobór ekwipunku bohatera

Cel i zadanie

Proszę dobrać ekwipunek bohatera przed wyprawą. Każdy z 12 dostępnych przedmiotów ma masę i dodatnią wartość użytkową, np. przydatność w danej misji. Plecak mieści najwyżej 15 kg, a każdy przedmiot można zabrać tylko raz. **Proszę maksymalizować sumę wartości bez przekroczenia limitu masy.** Proszę porównać wynik metaheurystyki z losowaniem i rozwiązaniem optymalnym.

Narzędzia i materiały startowe

Python, NumPy, Matplotlib i Jupyter Notebook; wystarcza CPU. Zespół liderski przygotowuje plik CSV z nazwami, masami i wartościami przedmiotów, wczytywanie danych, prezentację plecaka oraz szkielet algorytmu. Suma mas powinna przekraczać pojemność plecaka. Rozwiązanie opisuje wektor 12 zer i jedynek: jedynka oznacza zabranie przedmiotu.

Jako metaheurystykę proszę wykorzystać **symulowane wyżarzanie**. Wartości przedmiotów sumują się; interakcje między nimi są pomijane. Zespół liderski przygotowuje również funkcję pełnego przeglądu 4096 kombinacji do wyznaczenia optimum. Przegląd służy wyłącznie wyznaczeniu optimum, poza budżetem porównywanych metod.

Przebieg pracy przez 90 minut

0–10 min: proszę omówić reprezentację plecaka, cel i ograniczenie masy. Proszę otworzyć sprawozdanie i sprawdzić ręcznie dwa przykładowe zestawy.

10–25 min: proszę uzupełnić ocenę rozwiązania oraz metodę bazową losującą 500 wektorów. Dla przekroczonej masy proszę odrzucać zestaw, ale proszę wliczać próbę do budżetu. Proszę zapisywać najlepszą dopuszczalną wartość, zaczynając od pustego plecaka.

25–55 min: proszę uzupełnić symulowane wyżarzanie. Proszę zacząć od pustego plecaka; sąsiada proszę tworzyć przez zmianę jednego losowego bitu. Proszę odrzucać przeciążony plecak. Poprawę proszę przyjmować zawsze, a pogorszenie z prawdopodobieństwem zależnym od spadku wartości i temperatury, według reguły z konspektu. Stopniowo proszę obniżać temperaturę i zachowywać najlepszy znaleziony zestaw. Proszę wykonać 500 prób sąsiada, wliczając odrzucone propozycje.

55–75 min: proszę porównać obie metody dla pięciu ustalonych ziaren. Proszę zestawić wartość, masę i różnicę względem optimum. Proszę przedstawić na wykresie najlepszą wartość w funkcji liczby prób; proszę odróżnić ją od bieżącego rozwiązania, które może się pogarszać.

75–90 min: proszę uzupełnić wnioski, przygotować prezentację oraz przesłać kod i sprawozdanie na Teams. Notatki proszę prowadzić przez całe ćwiczenie.

Demo na forum grupy po części roboczej

5–6 minut na zespół: proszę pokazać wybrany ekwipunek, jego masę i wartość, porównanie z losowaniem oraz optimum. Proszę wyjaśnić jedną decyzję algorytmu i jeden zaobserwowany problem.

Co powinno znaleźć się w wyniku

Kod, tabela pięciu prób, wykres i parametry temperatury. Proszę wyjaśnić, po co algorytm dopuszcza chwilowe pogorszenie oraz czy znalazł optimum. Brak przewagi nad losowaniem jest dopuszczalnym wynikiem, który należy omówić.

Zakres dodatkowy

Proszę porównać dwa tempa chłodzenia przy tym samym budżecie prób i danych. Proszę nie zmieniać równocześnie pojemności plecaka, aby zachować czytelność porównania.

Warsztat 2 Agent uczący się przejścia labiryntu

Cel i zadanie

Proszę nauczyć agenta dotarcia do wyjścia na planszy **5 × 5 pól** z kilkoma ścianami. Proszę zastosować tablicowy Q-learning i sprawdzić, jak zmienia się skuteczność po uczeniu. Celem jest zrozumienie roli nagrody, eksploracji i oddzielenia treningu od oceny gotowej strategii.

Narzędzia i materiały startowe

Python, NumPy, Matplotlib i Jupyter Notebook; bez GPU i bez zewnętrznego silnika gry. Zespół liderski przygotowuje jedną osiągalną planszę, funkcje resetowania środowiska i wykonywania ruchu, rysowanie ścieżki oraz szkielet pętli uczenia. Stanem jest pozycja agenta, a akcjami ruchy w czterech kierunkach.

Proponowane nagrody: **+10 za wejście na cel, −0,1 za zwykły ruch i −1 za próbę wejścia w ścianę lub poza planszę**. Nagrody są rozłączne. Ruch niedozwolony pozostawia agenta na miejscu. Epizod kończy się po osiągnięciu celu lub po 50 krokach.

Przebieg pracy przez 90 minut

0–10 min: proszę sprawdzić zasady środowiska i obejrzeć przejście losowego agenta. Proszę otworzyć sprawozdanie i zapisać konfigurację.

10–25 min: proszę utworzyć tablicę wartości dla par stan–akcja. Proszę uzupełnić wybór epsilon-zachłanny: z prawdopodobieństwem epsilon proszę losować akcję, a w przeciwnym razie proszę wybierać największą wartość Q. Remisy proszę rozstrzygać losowo.

25–55 min: proszę uzupełnić aktualizację Q-learning. Dla osiągniętego celu wartość przyszła wynosi zero; w pozostałych przejściach proszę uwzględnić największą wartość Q kolejnego stanu. Proszę uruchomić 300 epizodów z losowymi, dozwolonymi startami, tempem uczenia 0,1, dyskontem 0,95 oraz epsilon 0,2. Gotowy szkielet obsługuje limit kroków odrębnie od osiągnięcia celu.

55–75 min: proszę porównać agenta losowego i wyuczonego na tej samej liście 20 przygotowanych pozycji startowych. Dla wyuczonego agenta proszę ustawić epsilon na zero i **proszę wyłączyć aktualizowanie tablicy Q podczas testu**. Proszę zapisać odsetek dotarć do celu oraz liczbę kroków w udanych próbach; proszę nie pomijać niepowodzeń w ocenie skuteczności.

75–90 min: proszę pokazać przykładową ścieżkę, podsumować obserwacje oraz przesłać kod i sprawozdanie na Teams.

Demo na forum grupy po części roboczej

5–6 minut na zespół: proszę pokazać przejście agenta przez labirynt, zestawić wynik z agentem losowym i wyjaśnić znaczenie eksploracji podczas uczenia.

Co powinno znaleźć się w wyniku

Tablica Q lub plik umożliwiający jej odtworzenie, parametry treningu, wykres nagrody w kolejnych epizodach i tabela testowa. Proszę wyjaśnić, dlaczego agent podczas treningu może wykonywać gorsze ruchy i dlaczego test z aktywną eksploracją nie mierzyłby samej wyuczonej strategii. Proszę zaznaczyć, że wynik dotyczy jednej planszy, a nie dowolnego labiryntu.

Zakres dodatkowy

Proszę porównać epsilon 0,05 i 0,3 przy tej samej liczbie epizodów, pozostałych parametrach i startach testowych. Proszę nie dodawać nowych mechanik gry kosztem rzetelnego porównania.

Warsztat 3 YOLO jako czujnik w minigrze

Cel i zadanie

Proszę przygotować prostą minigrę polegającą na pokazaniu wskazanego przedmiotu przed kamerą. **Wykrycie butelki w oznaczonej strefie obrazu odblokowuje nagrodę.** W podstawowym wariancie należy wykorzystać gotowe obrazy zamiast kamery. Celem jest połączenie wyniku sieci neuronowej z regułami gry i ocena wpływu progu ufności na błędne decyzje.

Narzędzia i materiały startowe

Python, Ultralytics YOLO, OpenCV i Jupyter Notebook. Zespół liderski wybiera mały, gotowy model detekcyjny, sprawdza obsługę klasy „bottle", pobiera wagi i przygotowuje zgodne wersje bibliotek. Model działa na CPU; jego trening nie jest częścią warsztatu. Skrypt startowy wczytuje obrazy i rysuje ramki detekcji.

Pakiet zawiera 12 niewielkich zdjęć: cztery z butelką w strefie, cztery z butelką poza strefą i cztery bez butelki. Każde zdjęcie ma referencyjną etykietę „nagroda" lub „brak nagrody". Zespół prowadzący wcześniej sprawdza obrazy, obejmując także trudniejsze przypadki, np. częściowe zasłonięcie. Należy wykorzystać rzeczywiste przedmioty należące do klas modelu, bez założenia, że rozpozna dowolne obiekty graficzne gry.

Przebieg pracy przez 90 minut

0–10 min: proszę omówić różnicę między klasą obiektu, ramką ograniczającą i wynikiem ufności. Proszę rozpocząć notatki.

10–25 min: proszę uruchomić detekcję na przykładowych zdjęciach. Proszę odczytać nazwę klasy, wynik ufności i współrzędne ramki. Proszę zdefiniować strefę jako środkową jedną trzecią szerokości obrazu, na całej jego wysokości.

25–55 min: proszę uzupełnić regułę gry: nagroda przysługuje, gdy co najmniej jedna wykryta butelka przekracza próg ufności, a środek jej ramki znajduje się w strefie. Na obrazie proszę zaznaczyć strefę i wynik decyzji. Jedno zdjęcie może dać najwyżej jedną nagrodę.

55–75 min: proszę uruchomić test wszystkich 12 obrazów dla progów **0,25 i 0,60**. Proszę zapisać poprawne nagrody, błędne nagrody, pominięte nagrody i poprawne odmowy. Proszę sprawdzić, czy przyczyną pomyłek jest brak detekcji, niepoprawna lokalizacja czy sama reguła gry.

75–90 min: proszę zapisać tabelę, dwa przykłady działania i wnioski. Proszę oddać kod oraz sprawozdanie przez Teams.

Demo na forum grupy po części roboczej

5–6 minut na zespół: proszę pokazać decyzję minigry na dwóch zdjęciach, porównać progi ufności i omówić jeden błąd detekcji lub przyznania nagrody.

Co powinno znaleźć się w wyniku

Nazwa modelu i wersje bibliotek, kod reguły decyzyjnej, zestawienie obu progów oraz zrzuty przykładu poprawnego i problematycznego. Proszę wyjaśnić koszt błędnego przyznania nagrody i pominięcia poprawnej akcji. Mały zestaw służy demonstracji; nie uzasadnia ogólnej oceny jakości detektora.

Zakres dodatkowy

Proszę uruchomić kamerę i przyznawać punkt dopiero po trzech kolejnych poprawnych detekcjach. Następny punkt proszę odblokować po opuszczeniu strefy, aby jeden nieruchomy przedmiot nie zwiększał wyniku w każdej klatce.

Warsztat 4 Rozmowa z kupcem sterowanym przez LLM

Cel i zadanie

Proszę zbudować tekstowy dialog z kupcem w grze. Gracz może pytać o towar i poprosić o zakup mikstury. **Model językowy rozpoznaje intencję i proponuje odpowiedź, a program rozstrzyga, czy transakcja jest dozwolona.** Celem jest poznanie sposobu połączenia swobodnej rozmowy ze ściśle określonym stanem gry.

Narzędzia i materiały startowe

Python, Jupyter Notebook, biblioteka json i przygotowane połączenie z LLM, np. przez Ollama. Model i środowisko muszą być uruchomione przed warsztatem. Zespół liderski zapewnia funkcję wywołującą model, obsługę timeoutu oraz prosty ekran tekstowy. Można użyć lokalnego, niewielkiego modelu instrukcyjnego lub uzgodnionej usługi. Zakres nie obejmuje treningu modelu ani budowy rozbudowanej gry.

Stan początkowy: gracz ma **15 monet**, kupiec dwie mikstury, każda kosztuje 10 monet. Dozwolone propozycje to „talk” i „buy_potion”; pojedyncza akcja zakupu dotyczy jednej mikstury. Model zwraca JSON z polami „reply” i „action”. Saldo, cena i zapas są utrzymywane przez program.

Przebieg pracy przez 90 minut

0–10 min: proszę omówić podział odpowiedzialności między model a logikę gry. Proszę rozpocząć sprawozdanie i zapisać nazwę modelu.

10–30 min: proszę uzupełnić instrukcję opisującą rolę kupca, aktualny stan i dozwolone akcje. Proszę zdefiniować format JSON; jeśli wybrane narzędzie obsługuje odpowiedzi zgodne ze schematem, proszę wykorzystać tę możliwość. Proszę sprawdzić zwykłe pytanie oraz prośbę o zakup.

30–55 min: proszę dodać walidację pól i listy dozwolonych akcji. Program sam sprawdza liczbę monet i zapas, a następnie wykonuje zakup albo go odrzuca. Niepoprawny JSON, nieznana akcja lub timeout nie zmieniają stanu. Komunikat potwierdzający transakcję powstaje na podstawie wyniku programu, nie samej wypowiedzi modelu.

55–75 min: proszę wykonać sześć prób: pytanie o ofertę; poprawny zakup; ponowny zakup bez wystarczających środków; zakup przy pustym magazynie; niejasną wypowiedź; żądanie nieobsługiwanej czynności. Proszę użyć przygotowanych stanów początkowych, z wyjątkiem drugiego zakupu, który kontynuuje wcześniejszą transakcję.

75–90 min: proszę zapisać odpowiedzi, proponowane akcje, decyzje programu i stany końcowe. Proszę podsumować obserwacje i oddać materiały.

Demo na forum grupy po części roboczej

5–6 minut na zespół: proszę zademonstrować zakup dozwolony i odrzucony. Proszę pokazać propozycję modelu oraz faktyczny stan gry i wyjaśnić rolę walidacji.

Co powinno znaleźć się w wyniku

Instrukcja dla modelu, kod walidacji, tabela sześciu testów oraz zwięzłe wnioski. Proszę oddzielić **poprawność formatu**, rozpoznanie intencji i poprawność zmiany stanu gry. Nawet odpowiedź zgodna ze schematem może zawierać błędną propozycję. Proszę podać ustawienia generowania; przy ponownym uruchomieniu tekst odpowiedzi może się różnić.

Zakres dodatkowy

Proszę dodać drugą postać o odmiennym sposobie wypowiadania się, zachowując identyczne reguły transakcji. Proszę sprawdzić, czy zmiana stylu wpływa na poprawność akcji.

Warsztat 5 Odporność postaci LLM na manipulację

Cel i zadanie

Proszę sprawdzić, czy gracz może nakłonić postać LLM do naruszenia zasad gry, np. przyznania nagrody bez ukończenia zadania. Proszę zbadać **prompt injection**, czyli próbę przemycenia instrukcji przez treść gracza lub tekst znaleziony w świecie gry. Celem jest odróżnienie niepożądanej odpowiedzi modelu od rzeczywistej, nieuprawnionej zmiany stanu gry.

Narzędzia i materiały startowe

Python, Jupyter Notebook, json i dostęp do LLM jak w warsztacie 4. Zespół liderski dostarcza gotowy program strażnika, niezależny od wcześniejszego ćwiczenia. Stan obejmuje flagi „quest_done", „reward_claimed" i saldo monet. Dozwolone akcje to rozmowa i próba odbioru nagrody.

Program zawiera celowo podatny wariant ufający modelowi i miejsce na walidator. Testy dotyczą lokalnej minigry, fikcyjnych danych i przygotowanych komunikatów. Nagroda wynosi 10 monet; przysługuje jednorazowo po ukończeniu zadania.

Przebieg pracy przez 90 minut

0–10 min: proszę sprawdzić regułę przyznawania nagrody i oba wejścia do modelu: wiadomość gracza oraz znalezioną notatkę. Proszę otworzyć sprawozdanie.

10–30 min: proszę sprawdzić sześć prób manipulacji, np. żądanie zignorowania reguł, podanie się za administratora i instrukcję w notatce. Proszę dodać trzy zwykłe przypadki: pytanie o zadanie, uprawniony odbiór nagrody i ponowną próbę odbioru. Każdy test należy zacząć od ustalonego stanu i nowej rozmowy.

30–55 min: proszę oznaczyć tekst gracza i notatkę jako dane oraz dodać instrukcję zachowania reguł i walidację schematu oraz akcji. Niezależnie od odpowiedzi modelu program sprawdza obie flagi, ustala kwotę i dopiero wtedy zmienia saldo. Tekst gracza nie może bezpośrednio nadpisywać flag.

55–75 min: proszę powtórzyć te same dziewięć przypadków. Proszę policzyć oddzielnie niedozwolone propozycje modelu, nieuprawnione zmiany stanu i błędne odrzucenia poprawnych próśb. Proszę porównać oba warianty przy tych samych stanach i ustawieniach modelu.

75–90 min: proszę opisać przyczynę jednego błędu lub próbę, która nie zadziałała. Proszę oddać kod, tabelę testów i sprawozdanie na Teams.

Demo na forum grupy po części roboczej

5–6 minut na zespół: proszę pokazać próbę manipulacji przed i po zabezpieczeniu, odróżnić propozycję modelu od zmiany stanu gry i przedstawić wyniki testów.

Co powinno znaleźć się w wyniku

Dziewięć testów, wyniki przed i po zmianie oraz opis walidatora. Brak udanej manipulacji nie gwarantuje bezpieczeństwa. Proszę też podać walidatorowi ręcznie przygotowaną niedozwoloną akcję, oznaczoną jako **test komponentu, a nie udany atak na LLM**.

Zakres dodatkowy

Proszę sprawdzić, czy strażnik obiecuje nagrodę mimo jej zablokowania przez program, i zaproponować komunikat zgodny ze stanem gry. Sama instrukcja dla modelu nie zastępuje kontroli uprawnień w kodzie.